

9. Recommender Systems

*“Without big data analytics,
companies are blind and deaf,
wandering out onto the web like deer on a freeway.”*
— Geoffrey Moore

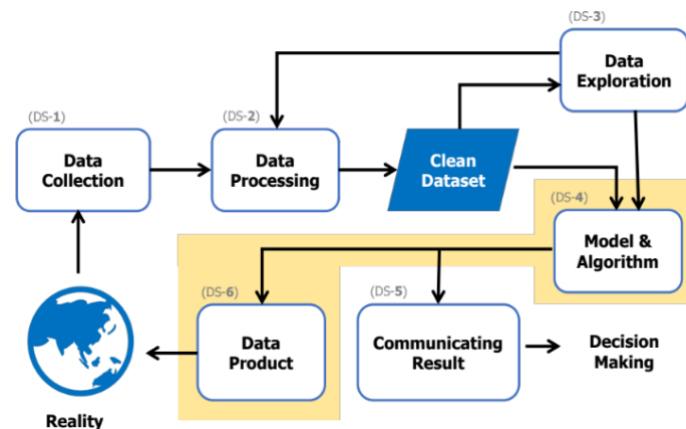


Figure 9.0-1: Scope of recommender systems in the data science process

A recommender system suggests options to a user in line with their goals — the fastest route, the cheapest route, a suitable course, an elective likely to yield a high grade, an insurance plan suited to retirement savings, a fund likely to profit long-term, a stock likely to profit short-term, a video the user is likely to watch, or a product the user is likely to buy.

Recommender systems are usually built as a feature within another system — for example, as part of an e-commerce app recommending products to customers. The idea: a good recommendation increases the chance a customer buys more, boosting sales and revenue; a poor recommendation wastes that opportunity. This makes recommender systems a key revenue strategy for online products and services.

A recommender system builds a prescriptive-analytics model and is a form of data product. This chapter follows the data science process stages shown in Figure 9.0-1, covering Model Development (DS-4) and Data Product (DS-6), and includes: the principles of recommender systems, evaluation methods, building a recommender with Collaborative Filtering, and the concept of Hybrid Recommender Systems.

9.1 Understanding Recommender Systems

A recommender system applies prescriptive analytics to build a model recommending things — products, websites, images, groups, pages, friends, and so on. These are collectively called items, presented to a user to choose from. “Choosing” is defined by the problem at hand — clicking a recommended website, watching a recommended video, listening to a recommended song, buying a recommended product, spending a long time engaging with something recommended, liking a recommended page, joining a recommended group, or adding a recommended friend (Ricci et al., 2011). Figure 9.1-1 shows Amazon.com's product page: (1) the user is viewing a gaming console; scrolling down reveals (2) related products and (3) items the user is likely to like.

An effective recommender system suggests items the user is genuinely likely to choose — a good system gets clicked, a poor one gets ignored. Good recommender systems therefore avoid recommending the same items to everyone, tailoring suggestions to each individual based on a prediction of what they'll choose.

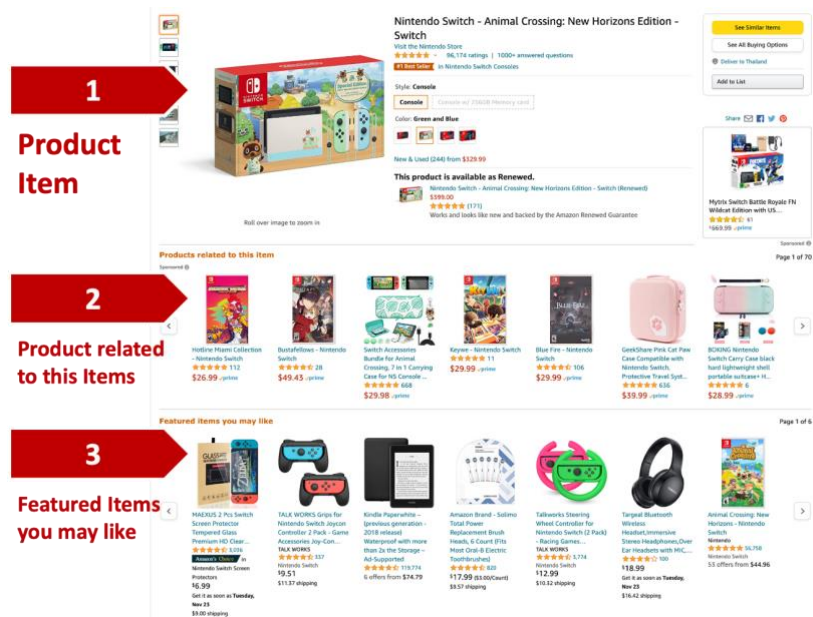


Figure 9.1-1: Example product recommendations on an e-commerce site (source: amazon.com)

Figure 9.1-2 shows the overall picture: a recommender system takes in various inputs — User Profile data, Collaborative Data (shared user behavior), Product Features, and a Knowledge Base — and builds a model that produces a Recommended List: predicted-but-not-yet-happened choices, made up of user, item, and score. A higher score means a higher predicted chance the user picks that item; the same item can score high for one user and low for another. The following sections cover the four recommendation approaches: by User Profile, by Collaborative Data, by Product Features, and by Knowledge Base.

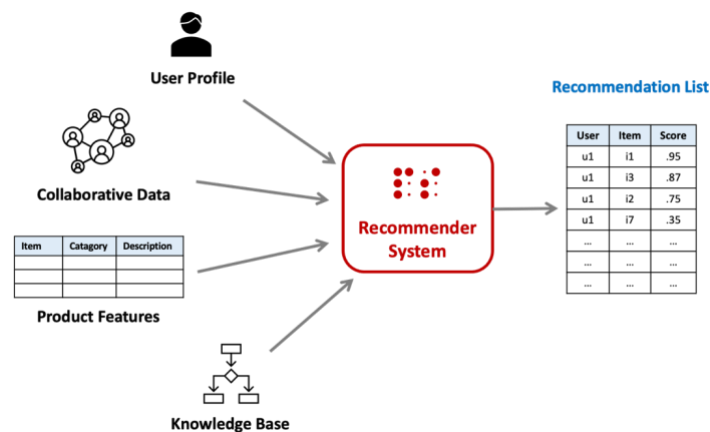


Figure 9.1-2: Overview of a recommender system

Recommend by User Profile

User profile data includes age range, gender, occupation, income, country, region, and so on. The idea: if several users with similar profiles (found via cluster analysis) chose a certain item, recommend that item to others with similar profiles. In Figure 9.1-3, users u1, u2, and u3 have similar profiles; since u2 and u3 bought item i1 but u1 hasn't, the system recommends i1 to u1.

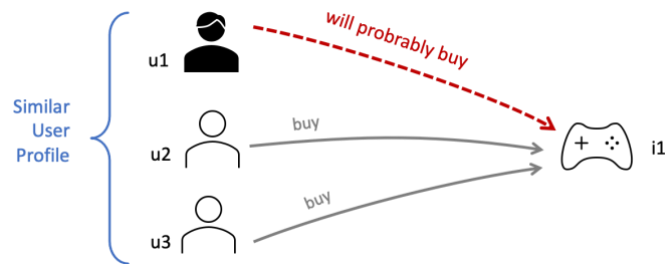


Figure 9.1-3: Example of recommending by user-profile similarity

Recommend by Collaborative Data

User profile data (age, gender, income, etc.) doesn't always predict similar behavior, so recommending based on shared item-selection behavior is another approach. In Figure 9.1-4, the system notes that u1, u2, and u3 all bought items i2 and i3, inferring similar buying behavior across these three users. Since u2 and u3 also bought i1 but u1 hasn't, the system infers u1 would likely be interested too, and recommends i1.

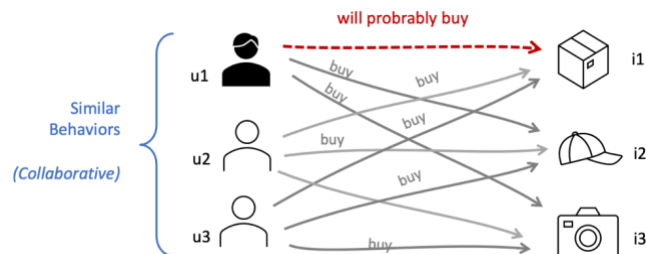


Figure 9.1-4: Example of recommending by shared item-selection behavior

Recommend by Product Features

Product features can include the manufacturer, category (appliances, kitchenware, tools, etc.), finer details (steam iron, mini oven), or a text description. The system scores item-to-item similarity, and if a user buys one item, recommends similar ones. In Figure 9.1-5, user u1 buys item i1; the system determines i2 is very similar to i1, and recommends i2 to u1.

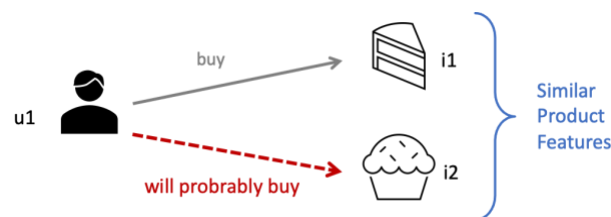


Figure 9.1-5: Example of recommending by item-feature similarity

Recommend by Knowledge Base

Recommending by item similarity suits frequently repurchased items (books, food, clothes, videos, music) but not items bought once and used for a long time (phones, TVs, fans, refrigerators, washing machines, irons) — a user who just bought an iron isn't likely to buy another soon, so recommending more irons makes little sense.

This calls for a Knowledge Base of recommendation rules based on domain expertise or data mining, as in Figure 9.1-6: if the knowledge base says “buying i1 suggests also buying i2,” then when user u1 buys i1, the system recommends i2.

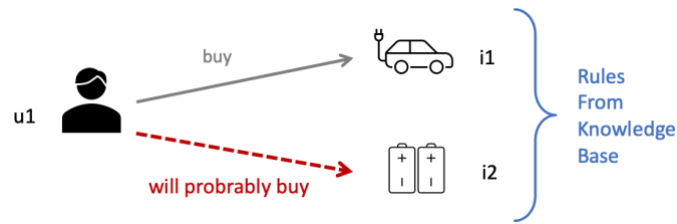


Figure 9.1-6: Example of recommending by knowledge base

9.2 Evaluating Recommender Systems

A recommender system's output — the Recommended List — predicts choices that haven't happened yet, made up of user, item, and score, as in Table 9.2-1. True effectiveness could be measured by actually presenting recommendations to real users and observing uptake (Jannach et al., 2010; Lü & Zhou, 2011), but this is costly in time and budget with uncertain payoff. In practice, evaluation instead uses held-out test data, via Train-Test Split or K-Fold Cross-Validation (see section 5.3). This chapter covers the held-out test-data approach, illustrated as follows:

- Suppose there are 12 items: {A, B, C, D, E, F, G, V, W, X, Y, Z}.
- User u1 has already bought 7 of them: {A, B, C, D, E, F, G}.
- So u1 hasn't bought 5 items: {V, W, X, Y, Z} — these are what the model scores to decide what to recommend.

To evaluate, suppose items A and B are held out as test data. The learning data then becomes:

- Purchased by u1 (for training): only 5 items — {C, D, E, F, G} — since A and B were removed.
- Not yet purchased by u1 (to be scored): now 7 items — {A, B, V, W, X, Y, Z} — treating A and B as if u1 never bought them.

The model is trained on u1's purchases of {C, D, E, F, G}, then scores the unpurchased set {A, B, V, W, X, Y, Z}, producing the ranked Recommended List in Table 9.2-1: user, item, predicted score (descending), and whether each item is held-out test data. If A and B (the test items) score well and appear near the top, the recommender is performing well; if they score poorly near the bottom, the system needs further improvement before real-world use.

Table 9.2-1: Example recommended list used for testing

User	Item	Predicted Score (desc.)	Test Data
u1	Z	0.8	
u1	A	0.7	test item
u1	B	0.6	test item
u1	Y	0.6	
u1	V	0.5	
u1	X	0.3	
u1	W	0.2	

Given this table of test-item recommendations, four evaluation methods are commonly used: MAE, Top-K Precision, hit@K, and AUC.

Mean Absolute Error (MAE)

MAE was covered in section 6.2.1; here only the formula is repeated for reference:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

where y_i is the value from the test data — suitable for measuring predicted item ratings (e.g., a 1–5 star rating) — and $\hat{y}_i$ is the score predicted by the recommender.

Top-K Precision

Top-K Precision picks the K highest-scoring items, then computes the ratio of test items found among them to K . This closely mirrors real-world use, where a fixed number of items (K) are shown to the user.

Principle

where K is the number of items recommended to the user, and N is the number of test items found among them:

$$\text{Top} - K \text{ Precision} = \frac{N}{K}$$

Worked Example

From Table 9.2-1, suppose the interface has room for 4 recommended items ($K=4$). The top 4 by score are $\{Z, A, B, Y\}$. Since A and B (the test items) both appear, $N=2$, giving:

$$\text{Top} - 4 \text{ Precision} = N/K = 2/4 = 0.5$$

This suits users with many past purchases, since more test items can be held out. In Table 9.2-1 only 2 items (A, B) were held out, so N can never exceed 2 no matter how large K gets — meaning larger K only drives Top-K Precision down as the denominator grows. When held-out data is limited, $\text{hit}@K$ (below) is preferable.

hit@K

$\text{hit}@K$ works like Top-K Precision but returns only 1 or 0: 1 if any test item appears among the top K recommendations, 0 otherwise — finding multiple test items still only scores 1. This suits datasets where each user has selected relatively few items.

Principle

$$\text{hit}@K = 1 \text{ if } \text{TEST} \cap \text{TOPK} \neq \emptyset; \text{ hit}@K = 0 \text{ otherwise}$$

where TEST is the set of test items and TOPK is the set of K recommended items; $\text{hit}@K$ is 1 if their intersection is non-empty, 0 otherwise.

Worked Example

With $K=4$, from Table 9.2-1, $\text{TEST} = \{A, B\}$ and $\text{TOPK} = \{Z, A, B, Y\}$. Since $\text{TEST} \cap \text{TOPK} = \{A, B\}$ is non-empty, $\text{hit}@4 = 1$.

Area Under the Curve (AUC)

Since Top-K Precision and $\text{hit}@K$ only look at the top K items, they answer practical business questions well — in practice, the business rarely cares about items ranked below the cutoff. But for research and model development, overall performance across the full ranking matters too: a good model should rank test items highly throughout the list, which is where AUC comes in.

Principle

The AUC formula, drawn from work on link prediction in complex networks (Lü & Zhou, 2011):

$$\text{AUC} = \frac{n' + 0.5n''}{n}$$

- n is the number of pairwise comparisons between test items and other recommended items.
- n' is the number of times a test item scored higher than another recommended item.
- n'' is the number of times a test item tied with another recommended item.

A good result should exceed 0.8 and approach 1, indicating test items rank consistently high among all recommended items.

Worked Example

From Table 9.2-1, the test items and scores are $\{A=0.7, B=0.6\}$, and the other recommended items and scores are $\{V=0.5, W=0.2, X=0.3, Y=0.6, Z=0.8\}$:

- n comes from all 10 pairs: (A,V), (A,W), (A,X), (A,Y), (A,Z), (B,V), (B,W), (B,X), (B,Y), (B,Z) — so $n=10$.
- n' comes from (A>V), (A>W), (A>X), (A>Y), (B>V), (B>W), (B>X) — 7 cases, so $n'=7$.
- n'' comes from (B=Y) — 1 case, so $n''=1$.
-

$$AUC = \frac{n' + 0.5n''}{n} = \frac{7 + 0.5 \times 1}{10} = 0.75$$

In practice: when the goal reflects a realistic business scenario (recommending a fixed number of items), use Top-K Precision or hit@K, supplemented with AUC for overall model quality. When the goal is purely rating prediction with no item selection involved, MAE alone suffices; but if the business goal is choosing which top-scoring items to recommend, that becomes a selection problem — use Top-K Precision or hit@K together with AUC instead.

9.3 Building a Recommender with Collaborative Filtering

Collaborative Filtering builds recommendations by filtering down to the group of users with similar item-selection or rating behavior (Ricci et al., 2011). This section covers the full process, from data preparation to calculation, using item-rating data, in an approach called User-Based Collaborative Filtering.

Data Preparation

Item ratings are typically 1–5 stars, with more stars meaning stronger preference. Rating data is usually stored as transaction records, as in Table 9.3-1 — raw data that isn't directly usable and must be converted into a User-Item Matrix, as in Table 9.3-2.

Raw Rating Data

Raw rating data typically takes a transactional form with key columns: User, Item, Rating, and Date_Time.

Table 9.3-1: Example raw per-user item-rating data

User	Item	Rating	Date_Time
Aom	item1	5	2021-01-14 00:01:06
user4	item1	1	2021-02-22 04:24:49
user1	item2	1	2021-03-10 05:50:57
Aom	item4	4	2021-05-11 00:36:50
user1	item5	3	2021-05-19 22:24:46
user2	item5	5	2021-06-04 10:09:06
user2	item6	4	2021-06-05 07:52:17
Aom	item3	4	2021-06-14 21:00:21
user3	item2	3	2021-09-05 23:34:32
...	...	...	...

User-Item Matrix

Table 9.3-1's data isn't directly usable for calculation and must be converted into a User-Item Matrix (Table 9.3-2), indexed by user (rows) and item (columns), with each cell holding that user's rating for that item. Empty cells (no rating given) are left blank (null/NA) or coded -1, by convention agreed on by the analytics team to avoid ambiguity in code. Here, for instance, Aom hasn't rated item5, so that cell is empty.

Table 9.3-2: User-item matrix of ratings

User	item1	item2	item3	item4	item5	item6
Aom	5	3	4	4		
user1	3	1	2	3	3	
user2	4	3	4	3	5	4
user3	3	3	1	5	4	
user4	1	5	5	2	1	1

Calculation

Collaborative Filtering's output is a predicted rating for every empty cell in the User-Item Matrix. In Table 9.3-2, this means computing scores for (Aom, item5), (Aom, item6), (user1, item6), and (user3, item6). The following works through (Aom, item5) as an example, in three steps:

1. Compute the correlation between Aom and every other user.
2. Select neighbors — other users with rating behavior similar to Aom.
3. Compute the rating Aom would likely give item5.

Computing Correlation

Correlation is computed as in section 4.3.1, using Pearson correlation, where $\text{Corr}(x, y)$ is the similarity between a pair of users — here, each other user compared against Aom:

$$\text{Corr}(x, y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

Only users who rated the same items as Aom, meeting some minimum overlap (e.g., at least 4 shared rated items), are included in the calculation.

For example, computing $\text{Corr}(\text{Aom}, \text{user1})$ uses only the items both rated — item1, item2, item3, item4 — giving Aom's values as [5, 3, 4, 4] and user1's as [3, 1, 2, 3]. Applying this to each other user gives:

- $\text{Corr}(\text{Aom}, \text{user1}) = 0.85$
- $\text{Corr}(\text{Aom}, \text{user2}) = 0.70$
- $\text{Corr}(\text{Aom}, \text{user3}) = 0$
- $\text{Corr}(\text{Aom}, \text{user4}) = -0.79$

Selecting Similar Neighbors

Users with high correlation to Aom become Aom's neighbors. With a threshold of 0.7, Aom's neighbors are {user1, user2} only.

Figure 9.3-1 illustrates this: user1 shows high correlation with Aom, rating items in the same direction (regardless of exact rating values), while user4's correlation of -0.79 shows a strongly opposite rating pattern — not a behaviorally similar neighbor.

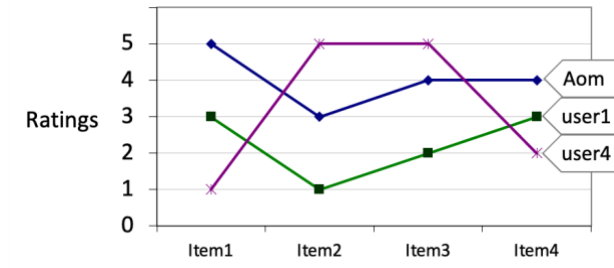


Figure 9.3-1: Example item-rating behavior across users

Computing the Predicted Score

Predicting the rating user u would give item i uses the following formula:

$$rec(u, i) = \overline{r_u} + \frac{\sum_{n \in N} sim(u, n) \times (r_{n,i} - \overline{r_n})}{\sum_{n \in N} sim(u, n)}$$

- $rec(u, i)$ the function predicting user u 's rating for item i
- $\overline{r_u}$ user u 's average rating across all rated items
- $\sum_{n \in N}$ N is the set of neighbors n who behave similarly to u and have also rated item i
- $sim(u, n)$ the similarity (correlation) between user u and neighbor n
- $r_{n,i}$ the rating neighbor n gave item i
- $\overline{r_n}$ neighbor n 's average rating across all rated items

Now, to predict Aom's likely rating for item5 via $rec(Aom, item5)$:

$$rec(Aom, item5) = \overline{r_{Aom}} + \frac{sim(Aom, user1) \times (r_{user1, item5} - \overline{r_{user1}}) + sim(Aom, user2) \times (r_{user2, item5} - \overline{r_{user2}})}{sim(Aom, user1) + sim(Aom, user2)}$$

- $\overline{r_{Aom}}$ $= (5+3+4+4) \div 4 = 4.0$
- $\sum_{n \in N}$ $N = \{user1, user2\}$ (the neighbors identified above)
- $sim(Aom, user1)$ $= 0.85$;
- $r_{user1, item5}$ $= 3$;
- $\overline{r_{user1}}$ $= (3+1+2+3+3) \div 5 = 2.4$
- $sim(Aom, user2)$ $= 0.70$;
- $r_{user2, item5}$ $= 5$;
- $\overline{r_{user2}}$ $= (4+3+4+3+5+4) \div 6 = 3.8$

Then

$$rec(Aom, item5) = 4.0 + \frac{0.85 \times (3 - 2.4) + 0.70 \times (5 - 3.8)}{0.85 + 0.70} = 4.87$$

Using User-Based Collaborative Filtering, Aom is predicted to rate item5 at 4.87 — effectively a 5-star rating.

Recommending Items

Once every empty cell of the User-Item Matrix has a predicted score, the highest-scoring items become each user's recommendations. The evaluation methods from section 9.2 can then be used to assess this approach with real data.

9.4 Building a Hybrid Recommender System

The four recommendation approaches — User Profile, Collaborative Data (via Collaborative Filtering), Product Features, and Knowledge Base — each have strengths and weaknesses depending on the data situation, summarized in Table 9.4-1.

Table 9.4-1: Comparing recommendation approaches

Technique	Advantages	Disadvantages
User Profile	Works without requiring learned behavior data; good for new users with no item-selection history	Users with similar personal data (age, gender, education, occupation) don't necessarily share item-selection behavior
Collaborative Data	Based on actual shared item-selection behavior, so realistic to real situations	Struggles with new users (Cold-Start Problem); can't recommend new, never-chosen items; can't recommend seasonally
Product Features	Can recommend similar items without learning selection behavior; gives new items a chance to be recommended	Not suited to items rarely repurchased (e.g., appliances); can't recommend seasonally
Knowledge Base	Can encode business rules directly; supports seasonal recommendations; works for new users; gives new items a chance	Doesn't analyze actual user behavior, so some recommendations may not be useful to the user

In practice, analysts and developers often combine approaches into a Hybrid Recommender System, using each technique's strengths to offset the others' weaknesses (Jannach et al., 2010). Each individual recommender model is called a scoring function, taking a user u and item i as parameters and returning u 's predicted score for i :

$$\text{scoring_function}(u, i) = \text{predicted score of user } u \text{ for item } i$$

where `scoring_function` is simply a name the developer chooses — e.g., $ABC(u, i)$, $JQK(u, i)$, $XYZ(u, i)$. A hybrid system then combines multiple scoring functions in one of three ways: Weighted, Switching, and Mixed.

Weighted

Combines scoring functions as a weighted sum. Given three scoring functions ABC , JQK , and XYZ , a hybrid recommender can be built as:

$$\text{HybridREC}(u, i) = a \cdot ABC(u, i) + b \cdot JQK(u, i) + c \cdot XYZ(u, i)$$

where a , b , and c are the weighting coefficients for ABC , JQK , and XYZ respectively, ideally chosen so that $a + b + c = 1$.

Switching

Chooses which scoring function to use based on conditions in the data, as in this pseudo-code:

```

FUNCTION HybridREC(u, i)
BEGIN
  IF (data satisfies condition_1) THEN
    return ABC(u, i)
  ELSE IF (data satisfies condition_2) THEN
    return JQK(u, i)
  ELSE
    return XYZ(u, i)
  END
END

```

Mixed

Uses each scoring function to power a different part of the system. For example, Figure 9.4-1 shows a video app's recommendation layout with three sections placed in the main interface: the first from scoring function ABC, the second from JQK, and the third from XYZ.

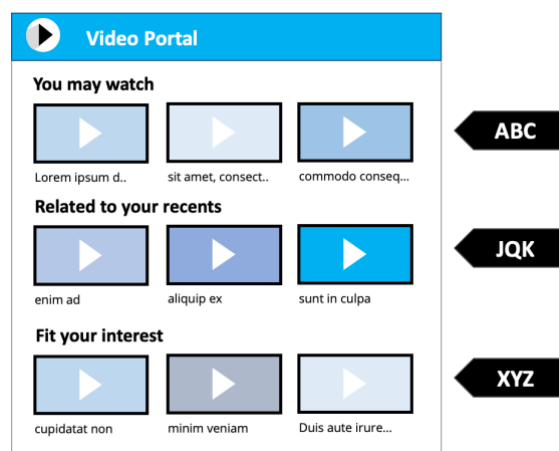


Figure 9.4-1: Example recommendation layout in a video application

9.5 Additional Technique: Matrix Factorization

Building a recommender with Collaborative Filtering requires computing and storing correlations for neighbor selection, which becomes very data-heavy for large-scale systems with huge numbers of users and items. To ease this, a model-based approach using Matrix Factorization can be used instead, which also helps with sparse datasets.

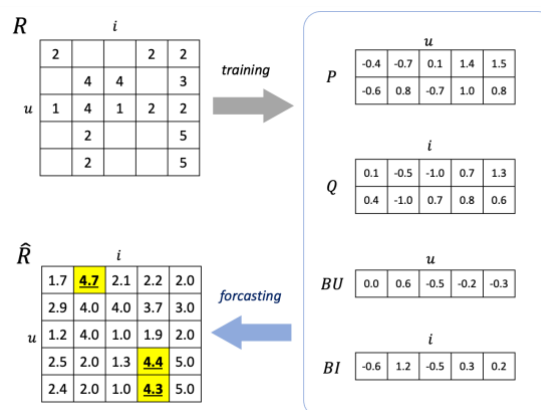


Figure 9.5-1: Building a recommender via matrix factorization

Matrix Factorization starts from the large User-Item matrix (call it R) and computes a reduced-dimension user matrix (feature matrix P) and item matrix (feature matrix Q), as in Figure 9.5-1. The following formula then predicts user u 's rating for item i :

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$$

- $\hat{r}_{ui}$ — the function predicting user u 's rating for item i (r_{ui} is the actual rating, where known)
- μ — the overall average rating
- b_u — user u 's bias term
- b_i — item i 's bias term
- p_u — the user vector, taken from feature matrix P at position u
- q_i — the item vector, taken from feature matrix Q at position i

In practice, b_u , b_i , p_u , and q_i are learned and refined, following the same approach as section 6.6 (the technical mechanics of linear regression). Since the recommender's output is a predicted rating — a regression problem — it likewise uses MSE as its loss function: $(\hat{r}_{ui} - r_{ui})^2$.

Each value is refined using Gradient Descent, as described in section 6.6, with the following per-iteration update rules:

$$b_i = b_i - \alpha \cdot (\hat{r}_{ui} - r_{ui})$$

$$b_u = b_u - \alpha \cdot (\hat{r}_{ui} - r_{ui})$$

$$q_i = q_i - \alpha \cdot (\hat{r}_{ui} - r_{ui}) \cdot p_u$$

$$p_u = p_u - \alpha \cdot (\hat{r}_{ui} - r_{ui}) \cdot q_i$$

Developers can also add further terms as needed — for example, a regularization term to reduce overfitting.

Implementing this in Python, using the R matrix from Figure 9.5-1 (with unrated cells set to -1), and randomly initializing vectors BU and BI along with feature matrices P and Q : MAE started at 1.72, and after 30 epochs of updates, dropped to 0.05. Some previously unrated user-item pairs then scored highly enough to be worth recommending — for example, $\hat{r}(0,1) = 4.7$, $\hat{r}(3,3) = 4.4$, and $\hat{r}(4,3) = 4.3$.

In the code below, lines 2–6 set up the initial data; lines 7–10 initialize and randomize the vectors and feature matrices; lines 11–17 perform the update loop; line 18 computes the full set of predictions; and line 19 measures MAE over only the previously rated cells.

```
import numpy as np

R = np.array([
    [ 2, -1, -1, 2, 2],
    [-1, 4, 4, -1, 3],
    [ 1, 4, 1, 2, 2],
    [-1, 2, -1, -1, 5],
    [-1, 2, 1, -1, 5]])

num_users, num_items = R.shape
num_features = 2
alpha = 0.1
m = R[R>-1].mean()
BI = np.random.rand(num_items)
BU = np.random.rand(num_users)
Q = np.random.rand(num_features, num_items)
P = np.random.rand(num_features, num_users)
for epoch in range(30):
    for u,i in np.argwhere(R>-1):
        r_pred = m + BI[i] + BU[u] + P[:,u].T.dot(Q[:,i])
        BI[i] = BI[i] - alpha*(r_pred-R[u,i])
        BU[u] = BU[u] - alpha*(r_pred-R[u,i])
        Q[:,i] = Q[:,i] - alpha*(r_pred-R[u,i])*P[:,u]
        P[:,u] = P[:,u] - alpha*(r_pred-R[u,i])*Q[:,i]
    R_PRED = m + BI[np.newaxis,:] + BU[:,np.newaxis] + P.T.dot(Q)
    mae = np.abs(R_PRED[R>0] - R[R>0]).mean()
    print(mae)
```

9.6 Chapter Summary

Recommender systems apply prescriptive analytics toward building data products. This chapter covered the fundamentals of recommending items a user is likely to want, via four approaches — User Profile, Collaborative Data, Product Features, and Knowledge Base — followed by evaluation methods (MAE, Top-K Precision, hit@K, and AUC), building a Collaborative Filtering recommender, and combining approaches into a Hybrid Recommender System via Weighted, Switching, and Mixed strategies, giving analysts and developers a foundation to build recommenders suited to their own problems and data.

9.7 Review Questions

4. Discuss how a recommender system differs from predictive analytics.
5. Discuss which approach(es) a recommender system should use if no user-profile data is available.
6. Discuss which evaluation method(s) would suit a route-recommendation system, and why.
7. Write Python code to convert Table 9.3-1 (raw per-user rating data) into Table 9.3-2 (the user-item ratings matrix), then compute `rec(Aom, item5)` using Collaborative Filtering as in section 9.3.
8. From Table 9.3-2, if Aom had rated item6 as 3, compute `rec(user1, item6)` using Collaborative Filtering as in section 9.3.
9. From Table 9.3-2, write Python code to build a recommender using Matrix Factorization, and compute the MAE of your model.
10. Study the movie-ratings dataset from Kaggle referenced at <https://github.com/Rathachai/DALAB/tree/gh-pages/exercises> (“Movie Ratings Dataset”), then write Python code to build a recommender using Matrix Factorization and compute MAE using 4-fold cross-validation.

9.8 References

- Hrnjica, B., Music, D., & Softic, S. (2020). Model-Based Recommender Systems. *Trends in Cloud-Based IoT*, 125–146.
- Jannach, D., Zanker, M., Felfernig, A., & Friedrich, G. (2010). *Recommender Systems: An Introduction*. Cambridge University Press.
- Lü, L., & Zhou, T. (2011). Link Prediction in Complex Networks: A Survey. *Physica A: Statistical Mechanics and Its Applications*, 390(6), 1150–1170.
- Provost, F., & Fawcett, T. (2013). *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O’Reilly Media, Inc.
- Ricci, F., Rokach, L., & Shapira, B. (2011). Introduction to Recommender Systems Handbook. In *Recommender Systems Handbook* (pp. 1–35). Springer, Boston, MA.
- Roy, D., & Dutta, M. (2022). A Systematic Review and Research Perspective on Recommender Systems. *Journal of Big Data*, 9(1), 59.
- Russell, S., & Norvig, P. (2002). *Artificial Intelligence: A Modern Approach*.